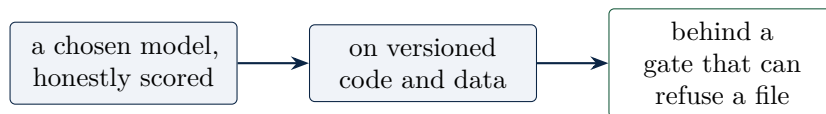


Machine Learning Operations

Lecture 08

Where Our Project Stands

Module I in one sitting



and still in a notebook, on purpose

This is a revision document, not a new lecture. Nothing here is new material. It is Module I, seven lectures, arranged so you can find any claim quickly and test whether you actually hold it.

Use it in this order: read Part I in ten minutes, then take the self test in Part IV *before* looking at the answers, and use what you get wrong to decide what to reread.

Contents

How this session works, and how it maps to the plan	4
I Module I in One Page	5
1 The spine of the story	5
2 The five claims, in five lines	5
3 The loop that explains the whole course	5
II The Seven Lectures, One Page Each	6
4 L01 and L02: the project and the promise	6
5 L03: the honest score	6
6 L04: the model, chosen by measurement	6
7 L05: the ledger for code	7
8 L06: the cloakroom for data	7
9 L07: the gate at the door	7
III The Reference Card	8
10 The numbers of Module I	8
11 The commands of Module I	8
12 The three things that travel with a model	9
13 Ten misconceptions worth checking you do not hold	9
IV Self Test	10
14 Twenty questions, no notes	10
V Answers	11
15 The answer key	11

VI This Week, and What Comes Next	12
16 Assignment questions, as released	12
17 What to post in the forum	12
18 What Module II will assume you already have	13
19 Where the project stands, and what happens to it next	14
VII Resource Library	15
20 The papers behind Module I	15
21 The manuals	15

List of Figures

1	Module I as one line. The dashed box is where Module II begins, and it begins by taking apart the notebook everything so far was built in.	5
2	The loop, from Lecture 01. Every stop on the semester arc exists to make one of these four boxes reliable.	5
3	The lineage triple, assembled across three lectures. This is what an auditor, a regulator or an incident review is actually asking for.	9
4	The readiness checklist for Module II. Every item is something a later lecture builds directly on, rather than something that would merely be nice to have.	13
5	The project after Module I, and the first move of Module II. The foundation stays; the container for the work changes.	14

How this session works, and how it maps to the plan

This is a Socio-Ed session, so there is no classroom hour. It has three parts, and the session plan specifies all three.

Part	What it is
Recap video	The released deck, narrated. Pause it wherever a claim surprises you
Discussion forum	One thread, <i>where our project stands</i> . Post, then reply to one classmate
Practice quiz	Covers Lectures 01 to 07. Non graded. It exists so that you know where you stand

The safety net, stated plainly

The quiz carries no marks. If the section's median falls below 60 percent, a remedial session is scheduled before Module II begins.

So there is nothing to gain by looking answers up, and something real to lose. A quiz you game tells the faculty that the section is ready when it is not, and the remedial session that would have helped you never happens.

One correction to the released deck, and it is the second time

The recap frame states that the random forest won in Lecture 04 and is the project's model. The measured table in Lecture 04 shows **linear regression at 1.92** minutes and the **random forest at 2.43** on the same test set.

The forest is the model the project *carries forward* from Lecture 11, for a stated engineering reason: it has settings worth tracking and produces an artefact worth registering. That is a different claim from winning.

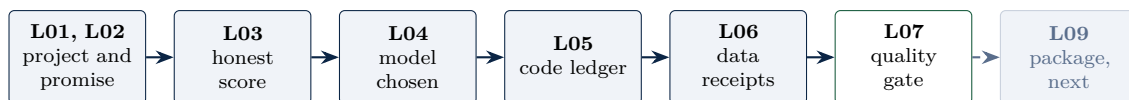
The same slip appears in the Lecture 05 recap. It is worth correcting in both decks, because Lecture 04's entire lesson was that we measure rather than assume, and a recap that reverses the result teaches the opposite.

PART I

Module I in One Page

1 The spine of the story

Seven lectures, one project, one argument. If you remember nothing else, remember this sequence and why each step had to come before the next.



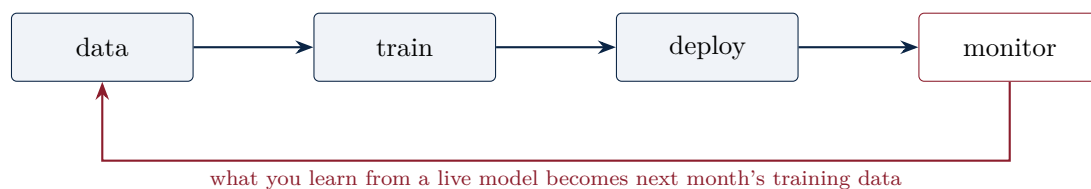
Each step is only possible because of the one before it. You cannot choose a model without an honest score; you cannot trust a score without versioned data; you cannot trust versioned data without a gate.

Figure 1: Module I as one line. The dashed box is where Module II begins, and it begins by taking apart the notebook everything so far was built in.

2 The five claims, in five lines

1. **One project, one promise.** The score was agreed before anything was built: mean absolute error, in minutes, lower is better.
2. **The lifecycle is a loop.** A model in a notebook is not a product, and monitoring feeds back into data.
3. **Git ledgers the code, DVC receipts the data,** and one pair of checkouts rebuilds any past state, rows and all.
4. **Cascades are avoidable,** upstream, at the gate.
5. **Data work is the model work.**

3 The loop that explains the whole course



The red arrow is the whole difference between a project and a product. Remove it and you have a straight line, which is a claim that the world holds still after you finish.

Figure 2: The loop, from Lecture 01. Every stop on the semester arc exists to make one of these four boxes reliable.

PART II

The Seven Lectures, One Page Each

4 L01 and L02: the project and the promise

The claim	A model that nobody can reach, trust or reproduce is not a production system
The evidence	Four documented failures: Zillow's \$407.9 million write down, Amazon's recruiting engine, Google Flu Trends, and 62 COVID imaging studies of which none were clinically usable
The three properties	Repeatable, reachable, watched. None of them mentions accuracy
To remember	A web server tells you when it crashes; a model does not tell you when it starts being wrong

5 L03: the honest score

The claim	A score computed on data the model has seen is a measure of memory, not ability
The evidence	Our own decision tree: 0.00 on train, 3.43 on test. Linear regression: 2.04 and 1.92
The trap	Leakage. Splitting after scaling means the test rows influenced the training, and the score is a lie
To remember	If you never hid any data, you do not have a score

6 L04: the model, chosen by measurement

The claim	The fanciest model is not automatically the best, and which one wins on your data is a measurement rather than an opinion
The evidence	Six models, one split: baseline 10.32, linear regression 1.92, forest 2.43, nearest neighbours 2.91, tree 3.22, support vector 3.70
The subtlety	Linear regression won; the project carries the forest from Lecture 11 for a stated engineering reason. Two different claims
To remember	Our simplest model won, and we published that

7 L05: the ledger for code

The claim	Copies are not history. A commit records what changed, by whom, when, and why
The mechanism	A commit holds a snapshot, a parent pointer, an author, a time, a message and a hash computed from all of it
Why it matters here	Because the hash is computed from the content, a model artefact recording a commit hash can be traced to exactly the code that built it
To remember	If it is not committed, it does not exist

8 L06: the cloakroom for data

The claim	Git keeps every version forever, which is right for code and fatal for data
The mechanism	A five line receipt naming the file by fingerprint. Committing the receipt pins that exact data to that commit
The measured demonstration	ccd4653d... at 10783 bytes and 600 rows, then 71a3d897... at 11542 bytes and 650 rows
To remember	Git keeps the receipt, DVC keeps the luggage, and the two checkouts always travel together

9 L07: the gate at the door

The claim	A perfectly versioned dataset can still be perfectly bad, and a great score on bad data measures nothing
The evidence	53 practitioners interviewed, 92 percent had lived through a cascade and 45.3 percent through two or more
The mechanism	The schema is a contract: columns, then types, then ranges, checked on every arrival
To remember	Files from outside are guests, not family. Data work is the model work

PART III

The Reference Card

10 The numbers of Module I

If a number appears in this table, it came from a real run or a cited paper. None of them is illustrative.

What	Value	From
Dataset, version 1	600 rows, 10783 bytes	L06
Dataset, version 2	650 rows, 11542 bytes	L06
Train and test split	480 and 120 rows	L03
Baseline, always guess the average	MAE 10.32	L04
Linear regression	MAE 1.92, RMSE 2.48	L03, L04
Random forest, 150 trees	MAE 2.43	L04
Decision tree, unlimited depth	0.00 train, 3.43 test	L03
Practitioners interviewed	53	L07
Experienced at least one cascade	92 percent	L07
Experienced two or more	45.3 percent	L07
GitHub warning, then hard block	50 MiB, then 100 MiB	L06

11 The commands of Module I

The ledger

```
git status
git add train.py
git commit -m "why, not what"
git log --oneline
git switch -c faster-model
git push -u origin faster-model
```

The cloakroom

```
dvc init
dvc add data/delivery_times.csv
git add data/*.dvc data/.gitignore
git commit -m "Version the dataset"
git checkout <commit>
dvc checkout
```

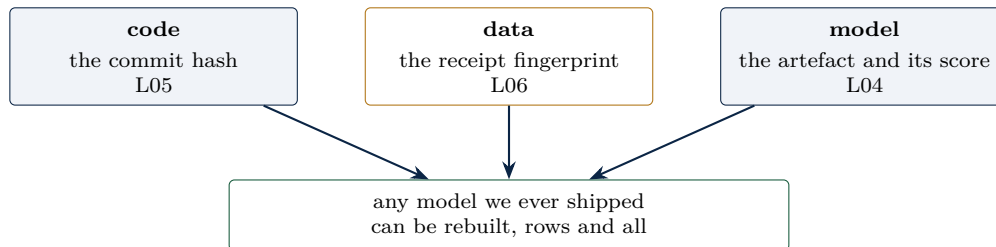
The two pairings you must not break

Code: `git add` then `git commit`. Staging without committing records nothing.

Data: `git checkout` then `dvc checkout`. Moving one without the other leaves the ledger and

the cloakroom disagreeing, and every score computed in that state is meaningless.

12 The three things that travel with a model



Remove any one of the three and the claim collapses. From Lecture 11 the tracking server records all three automatically, and it can only do so because you versioned both halves by hand first.

Figure 3: The lineage triple, assembled across three lectures. This is what an auditor, a regulator or an incident review is actually asking for.

13 Ten misconceptions worth checking you do not hold

Sounds right	Is wrong because
A high accuracy means the model is good	Accuracy is a statement about a model and a dataset, not about the world
The test set tells me which model to pick	Using it to choose is using it to train. Choose on cross validation, evaluate once
Overfitting is a property of an algorithm	It is observed behaviour: two numbers that disagree
A perfect training score is an achievement	It is a symptom
The forest won in Lecture 04	Linear regression did, at 1.92 against 2.43
Deleting a committed file shrinks the repository	Git keeps every earlier version. The blob is still in the history
<code>.gitignore</code> protects data I already committed	It only affects future commits
A fingerprint proves the file was not tampered with	It detects change. It is not a security measure
<code>git checkout</code> restores an old dataset	It restores the receipt. <code>dvc checkout</code> restores the file
Coercing bad cells fixes the file	It converts a loud error into a silent one and destroys the evidence

PART IV

Self Test

14 Twenty questions, no notes

Answer these before looking at Part V. Give yourself one mark each, and use the result to decide what to reread rather than to feel anything in particular.

1. Name the three properties that define MLOps.
2. What flows backwards in the lifecycle loop, and from where to where?
3. Why is a delivery time prediction a regression problem and not a classification problem?
4. State the formula for mean absolute error in words.
5. Two models have the same MAE but very different RMSE. What does that tell you?
6. Our decision tree scored 0.00 on train and 3.43 on test. Name the phenomenon, and say which number belongs in a report.
7. Give the correct order of splitting and scaling, and name the error made by the other order.
8. What does `random_state=42` buy you, and what argument does it prevent?
9. In the six model comparison, which model won and by how much over the baseline?
10. Why does the project carry a random forest from Lecture 11 anyway?
11. Name the five things a commit contains.
12. Why is a commit hash a trustworthy reference to a piece of code?
13. You are on `main` and your push is rejected. What is Git protecting, and what do you run?
14. Give two files that belong in the repository and two that do not.
15. What three things does `dvc add` do, and which one do you commit?
16. Describe the state of the project after `git checkout` but before `dvc checkout`.
17. Define a data cascade, and name the two properties that make one hard to catch.
18. Give the four triggers of a cascade.
19. State the difference between goodness of fit and goodness of data.
20. Your gate fails on the vendor's file. Name the three honest responses and the one that is not.

How to read your score

16 or more. You are ready for Module II. Post something in the forum that another student will find useful.

11 to 15. Reread the lectures behind the questions you missed, then retake it. This is the normal result at this point.

10 or fewer. Do not skip ahead. Module II assumes every one of these, and the readiness checklist in Part VI is the fastest route back. Say so in the forum; you will not be the only one.

PART V

Answers

15 The answer key

1. Repeatable, reachable, watched.
2. Monitoring feeds back into data. What you learn from a live model becomes the next training data.
3. The answer is a number on a scale, in minutes, rather than a group label.
4. The average size of the error, taking every error as positive, in the units of the target.
5. RMSE squares the errors first, so a large gap means a few very bad predictions rather than many small ones.
6. Overfitting. The test number, 3.43, is the one that belongs in a report.
7. Split first, then fit the scaler on the training rows only. The other order is data leakage.
8. It fixes the shuffle, so two people score on the same rows. It prevents arguing about models when only the split changed.
9. Linear regression, at MAE 1.92 against the baseline's 10.32.
10. Because it exercises the machinery the course is about: settings worth tracking, and an artefact worth registering.
11. A snapshot, a parent pointer, an author, a time, a message, and a hash computed from all of them.
12. Because the hash is computed from the content, so the same hash always means the same code.
13. It is protecting commits it can see and you cannot. Run `git pull`, resolve any conflict, then `git push`. Never force.
14. In: source files, the requirements file, the README, configuration. Out: the dataset, trained models, the environment folder, notebook outputs.
15. It moves the contents into the cache, writes the receipt, and adds the raw file to the ignore file. You commit the receipt.
16. The receipt is at the old version and the file on disk is still at the new one. The two halves disagree and any score computed there is meaningless.
17. Compounding events caused by data issues whose negative effects appear downstream. They are opaque and delayed.
18. Physical world brittleness, missing domain expertise, conflicting incentives, poor documentation.
19. Goodness of fit is how well the model matches the data it was given. Goodness of data is how far that data deserves to be trusted at all.
20. Quarantine the file, alert a human, stop the run. Never silently drop the bad rows.

PART VI

This Week, and What Comes Next

16 Assignment questions, as released

Set 1 • Production mindset • CO1

Q1 (Understand). Explain why a model in a notebook is not yet a product, and why the lifecycle is drawn as a loop rather than a straight line. Name what flows backwards.

Q2 (Apply). A team builds a churn model first and debates the success metric afterwards, in the demonstration meeting. Using the promise first rule, explain in three sentences what they did wrong and what should have happened.

Q3 (Build). Re run your model comparison from the machine learning bridge lectures on the *current* version of the dataset. Post your MAE table to this week's forum thread, naming which model you would ship and why.

Set 2 • Versioning code and data • CO1

Q1 (Understand). Contrast what Git stores with what DVC stores for our project, and explain why the receipt file solves the problem of Git ballooning on data.

Q2 (Apply). Your history shows commit `a1b2c3d`, dataset version 1 with 600 rows, and commit `e4f5g6h`, dataset version 2 with 650 rows. You are on version 2. Write the exact command sequence that puts the 600 row dataset back on disk, then returns to version 2.

Q3 (Build). In your repository, add a table to the README mapping each dataset version to its commit and row count. Fill it by actually switching versions with the checkout pair and counting rows, and paste the two counts as evidence.

Set 3 • Data foundations • CO1

Q1 (Understand). State the difference between goodness of fit and goodness of data, and give one example measure of each for the delivery time project.

Q2 (Apply). A vendor file arrives with one `rain` cell containing the text `N/A`. Predict what `df.dtypes` shows for that column, and name which check of the gate, columns, types or ranges, fires first.

Q3 (Build). Add one new check to your `gate()`: `delivery_min` between 5 and 180 minutes. Break a copy of the file to prove the check fires, and post the failing assertion line to the forum thread.

17 What to post in the forum

One thread, *where our project stands*. A post that helps somebody contains one of these.

Post this	Because
Your MAE table on the current data	Other people can check their numbers against yours, and disagreements are interesting
Your dataset version table	It proves the checkout pair worked on your machine, which is the commonest thing to get stuck on
One thing you still find shaky	Somebody else has the same gap and has not said so
A reply to one classmate	Confirm their claim, or politely break it. Both are useful

The most valuable post in any consolidation thread

Not the polished result. The one that says: I ran the checkout pair and got 650 rows both times, here is what I typed. Somebody answers it in ten minutes, and four other people who had the same problem stop being stuck.

18 What Module II will assume you already have

Lecture 09 begins by taking the notebook apart. It assumes, without checking, that all of this is already true on your machine.

A working environment

Step 0 prints All good, Step 1 reports Inside .venv True

A repository with real history

several commits, messages that say why

The dataset under DVC

a committed receipt, and both checkouts tried

A gate you have seen fail

not just one you have seen pass

A model and a number

and a written reason for choosing it

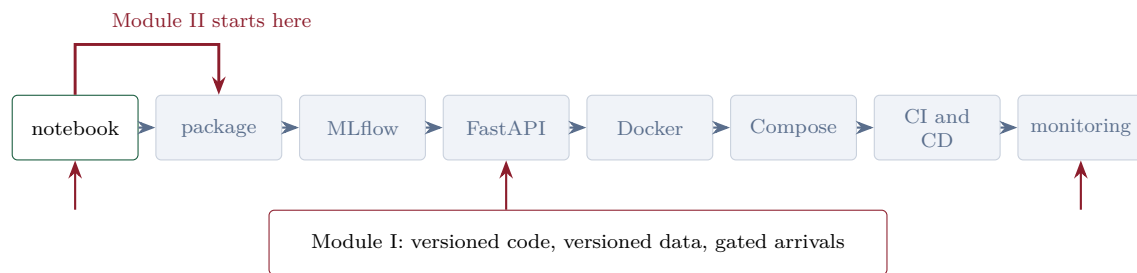
A notebook you can rerun

from the first cell, in a clean kernel

Anything on this list that is not true is cheaper to fix this week than in week nine, when four more layers are sitting on top of it.

Figure 4: The readiness checklist for Module II. Every item is something a later lecture builds directly on, rather than something that would merely be nice to have.

19 Where the project stands, and what happens to it next



A chosen model with an honest score, on versioned code, versioned data and gated arrivals. Still in a notebook, on purpose, because that notebook is exactly what Module II begins by breaking apart.

Figure 5: The project after Module I, and the first move of Module II. The foundation stays; the container for the work changes.

PART VII

Resource Library

All links verified on 11 September 2026. Nothing here is new: these are the sources behind the seven lectures, gathered in one place for revision.

20 The papers behind Module I

Source	Where, and which lecture it belongs to
Sambasivan et al., Data Cascades in High Stakes AI, CHI 2021	https://doi.org/10.1145/3411764.3445518 L07, and the reason Module I ends with a gate
Beede et al., A Human Centered Evaluation of a Deep Learning System Deployed in Clinics, CHI 2020	https://doi.org/10.1145/3313831.3376718 L07, the clinic story
Sculley et al., Hidden Technical Debt in Machine Learning Systems, NeurIPS 2015	https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fc2674f757a2463eba-Paper.pdf L01, where the problem was named
Kapoor and Narayanan, leakage and the reproducibility crisis, Patterns 2023	https://doi.org/10.1016/j.patter.2023.100804 L03, why leakage is not a student problem
Sato, Wider and Windheuser, Continuous Delivery for Machine Learning	https://martinfowler.com/articles/cd4ml.html L05 and L06, the three axes of change
Amershi et al., Software Engineering for Machine Learning, ICSE 2019	https://www.microsoft.com/en-us/research/wp-content/uploads/2019/03/amershi-icse-2019_Software_Engineering_for_Machine_Learning.pdf L01, the nine stage workflow

21 The manuals

Tool	Where
Pro Git, chapters 1 to 3, free	https://git-scm.com/book/en/v2
Get Started with DVC	https://dvc.org/doc/start
pandas user guide, dtypes	https://pandas.pydata.org/docs/user_guide/basics.html

Tool	Where
scikit-learn, common pitfalls and recommended practices	https://scikit-learn.org/stable/common_pitfalls.html
scikit-learn, metrics and scoring	https://scikit-learn.org/stable/modules/model_evaluation.html
Google, Rules of Machine Learning	https://developers.google.com/machine-learning/guides/rules-of-ml

Foundations done. Now we engineer.

Next lecture. Lecture 09, out of the notebook. Module II begins with code somebody else can run: why notebooks stop working as projects grow, refactoring into functions and classes, and the folder structure the project keeps for the rest of the semester. Laboratory 4 comes first, recording the baseline the semester will improve.